
AR-Retrofit: Retrofitting Pretrained Decoders with Attention Residuals for Adaptive Depth

Anonymous Author(s)

Affiliation

Address

email

Abstract

Adaptive computation is a central principle of biological intelligence: simple stimuli can be handled by shallow feedforward processing, whereas difficult stimuli recruit deeper and recurrent computation. For Transformers, a practical route toward this principle is to make depth composition input dependent. Attention Residuals (AttnRes) do so by letting each block attend to previous block outputs, replacing fixed residual accumulation with routed depth aggregation and improving representation quality. However, they usually require adopting the AttnRes architecture during training and introduce extra inference cost. We propose AttnRes Retrofit (AR-Retrofit), a lightweight post hoc method that converts an already trained decoder into an AttnRes-style model without updating backbone weights. It inserts a γ -gated residual correction path between frozen blocks, adding and training less than 0.4% new parameters on Qwen3-VL-2B. Unlike standard PEFT modules, this path is conditioned on learned depth aggregation over previous block states, yielding both a representation update and a routing signal. We further propose Routing-guided Residual Skipping (ReSkip), which uses the learned routing weights as pre-execution block-importance signals, calibrates them with offline safety sweeps, and skips low-contribution blocks at inference time to recover the cost introduced by AttnRes. On Qwen3-VL-2B and Qwen3-VL-4B, AR-Retrofit improves six VLM benchmarks by +2.1 and +1.2 on average, and raises LAMBADA accuracy by +3.3 and +8.7. When used to initialize LIBERO VLA policies, retrofitted backbones improve matched OFT baselines by +1.3 and +0.7; with ReSkip, they preserve scores while recovering much of the added inference cost. Under matched inference settings, the full-depth retrofit adds only small overhead in inference, while dynamic skipping provides a separate latency lever. These results show that trained decoders can be converted into practical AttnRes-style models at low cost, gaining routed depth aggregation and calibrated adaptive depth inference.

1 Introduction

Different inputs should not always consume the same amount of computation, an intuition supported by cognitive theories and biological perception [18, 24, 7, 19]: simple stimuli can often be resolved with fast feedforward processing, while difficult or ambiguous inputs recruit deeper, recurrent computation. Modern large models already reflect this principle along the token axis: test-time compute scaling, chain-of-thought prompting, and self-thinking methods allocate more computation to harder problems by generating additional intermediate tokens [32, 13, 37, 42, 45]. Yet, once a token is generated, it still passes through a fixed sequence of Transformer blocks. Consequently, for LLMs, VLMs, and VLA models, depth remains a complementary axis of adaptive computation.

Existing depth-adaptive methods mostly focus on which blocks should be executed. Early-exit methods [36] attach prediction heads to intermediate layers; Mixture-of-Depths [35] learns token-level routers; LayerSkip [10] modifies the training recipe to support early exiting and self-speculative decoding; and post-hoc layer pruning [9, 12, 31] removes layers after training. These methods are effective for improving efficiency, but their depth decisions typically rely on auxiliary prediction modules, explicit routers, special training objectives, or static pruning rules, and often require additional training, calibration, or task-specific tuning. A complementary direction is to make the representation itself depth-adaptive. Attention Residuals [40] (AttnRes) replace fixed residuals with attention over previous block outputs, letting each block dynamically aggregate depth information and produce an internal routing signal. Existing AttnRes models, however, are trained from scratch and incur extra inference overhead. This creates a practical gap: most LLM, VLM, and VLA backbones are pretrained standard-residual decoders, making full AttnRes retraining impractical. Therefore, we ask: *can a pretrained model be cheaply converted to support depth-adaptive representations and computation, dynamically combining information and adjusting computation across layers?*

To this end, we propose Attention-Residual Retrofit (AR-Retrofit), a lightweight post-hoc retrofit method that equips frozen pretrained decoders with both representation-level and execution-level depth adaptivity. AR-Retrofit introduces AttnRes-style depth aggregation by inserting a small γ -gated routing residual correction path between frozen blocks, leaving backbone weights unchanged. The retrofit path learns to route over previous block outputs and injects a gated correction into the original computation stream, preserving the pretrained backbone while adding less than 1% trainable parameters. Unlike conventional PEFT, this correction leverages earlier block states and exposes a pre-execution depth signal.

To translate the learned routing into adaptive computation, we introduce Routing-guided Residual Skipping (ReSkip). ReSkip calibrates the routing weights into input-dependent block-importance signals and combines them with offline skippable-position selection to skip low-contribution blocks under a constrained budget. In this way, the same routing mechanism both enhances depth-wise representation aggregation and recovers most of the extra inference cost introduced by AttnRes.

We evaluate AR-Retrofit on LLM, VLM, and VLA benchmarks using Qwen3-VL-2B/4B. It improves multimodal and language performance with minimal trainable parameters. When used to initialize LIBERO VLA policies, retrofitted backbones outperform matched OFT baselines. ReSkip recovers much of the extra inference cost while revealing the quality-latency tradeoff. These results show that pretrained decoders can be post-hoc converted into practical adaptive-depth models without full retraining or backbone updates. Our contributions are as follows:

- **AR-Retrofit:** a post-hoc method converting pretrained decoders into AttnRes-style models via a small γ -gated residual correction path, keeping backbone weights frozen; On Qwen3-VL-2B, AR-Retrofit introduces less than 0.4% new parameters.
- **ReSkip:** calibrates the routing weights from AR-Retrofit into input-dependent block-importance signals, combined with offline skippable-position selection for dynamic block skipping.
- Evaluated on Qwen3-VL-2B/4B across LLM, VLM, and LIBERO VLA benchmarks; AR-Retrofit improves performance, while ReSkip provides a calibrated latency-quality tradeoff.

2 Method

Our method has two components: AR-Retrofit inserts an AttnRes-conditioned residual path into a frozen decoder, and ReSkip converts the resulting phase-1 routing weights into calibrated block-skip decisions. Both rely on the AttnRes signal, defined next.

Preliminary: Attention Residuals. A standard transformer uses fixed scalar-1 combination, $\mathbf{x}_l = \mathbf{x}_{l-1} + f_l(\mathbf{x}_{l-1})$, emitting no routing signal. AttnRes [40] replaces this with a learned attention over depth. Each block l maintains a pseudo-query $\mathbf{w}_l \in \mathbb{R}^d$; keys are projections $\mathbf{k}_i = W_K \mathbf{h}_i$ of earlier block outputs $\mathbf{h}_i$, and

$$\alpha_{i \rightarrow l} = \frac{\exp(\mathbf{w}_l^\top \mathbf{k}_i / \sqrt{d})}{\sum_{j=0}^{l-1} \exp(\mathbf{w}_l^\top \mathbf{k}_j / \sqrt{d})}, \quad \mathbf{x}_l = \sum_{i=0}^{l-1} \alpha_{i \rightarrow l} \mathbf{h}_i. \quad (1)$$

The key property is two-phase execution. Computing the input to block l requires only $\{\mathbf{h}_i\}_{i < l}$ and $\mathbf{w}_l$: both available *before* f_l runs. We call the pre-execution attention computation *phase 1* and f_l

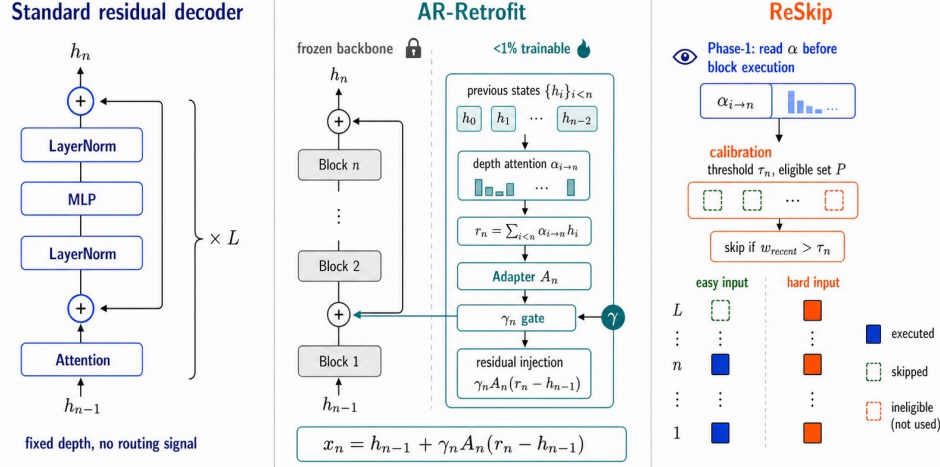


Figure 1: **Overview of AR-Retrofit and ReSkip.** (a) A standard residual transformer provides no intrinsic depth-routing signal. (b) AR-Retrofit inserts γ -gated AttnRes routing into a frozen pretrained backbone, yielding a pre-execution signal. (c) ReSkip reads phase-1 α to calibrate input-dependent block execution: easy inputs skip more blocks, while hard inputs retain more depth.

87 itself *phase 2*. Phase 1 is the routing signal we exploit. Block-AttnRes groups layers into N blocks
 88 and applies the softmax at the block level, which is the granularity used throughout. Original AttnRes
 89 models adopt this residual structure during training; here we retrofit a pretrained standard-residual
 90 decoder while leaving its backbone weights frozen.

91 2.1 AR-Retrofit: installing routed depth into a frozen decoder

92 The retrofit targets a standard-residual pretrained transformer with frozen weights. The added path
 93 must be identity-preserving at initialization, place the AttnRes routing signal on the forward path,
 94 and provide a meaningful surrogate when a block is skipped. This rules out simpler variants: local
 95 adapters [14, 15] lack depth routing; observer-only routers [35] do not affect representations; and
 96 direct AttnRes replacement [40] can shift frozen blocks off their pretrained manifold.

97 Let D be the number of decoder layers and group them into N blocks of L_{blk} layers each, so
 98 $N = D/L_{\text{blk}}$. For every block $n \in \{1, \dots, N\}$ we add an AttnRes router, a bottleneck residual
 99 adapter A_n , and a scheduled scalar gate γ_n . The method notation is one-based; empirical block IDs
 100 reported in tables and diagnostics use the zero-based implementation convention. Given completed
 101 block states $\{h_i\}_{i=0}^{n-1}$, where h_0 is the embedding stream, the retrofit router computes

$$\tilde{k}_i = \text{RMSNORM}(h_i) + b_i, \quad \alpha_{i \rightarrow n} = \frac{\exp(\mathbf{w}_n^\top \tilde{k}_i / \sqrt{d})}{\sum_{j=0}^{n-1} \exp(\mathbf{w}_n^\top \tilde{k}_j / \sqrt{d})}, \quad r_n = \sum_{i=0}^{n-1} \alpha_{i \rightarrow n} h_i. \quad (2)$$

102 Here $b_i \in \mathbb{R}^d$ is a learned source bias. The routed state is not substituted directly into the frozen
 103 backbone. Instead, it enters as a gated low-rank correction around the pretrained block input:

$$x_n = h_{n-1} + \gamma_n A_n(r_n - h_{n-1}), \quad h_n = \text{Block}_n(x_n). \quad (3)$$

104 The term $r_n - h_{n-1}$ is the routed-depth deviation from the ordinary residual predecessor. The
 105 adapter maps it into a low-rank correction, while the scalar schedule $\gamma_n(t)$ controls how strongly
 106 that correction enters the frozen block. The gradual gating ensures the retrofit adapts smoothly,
 107 avoiding sudden shifts that could break the frozen backbone. Particularly, the pretrained block
 108 group Block_n remains unchanged; AttnRes is inserted only between blocks. Each adapter, $A_n(u) =$
 109 $W_n^{\text{up}} \sigma(W_n^{\text{down}} u)$ with bottleneck rank r , adds $2dr$ trainable weights. Only routers and adapters are
 110 optimized; scalar gates follow a deterministic curriculum. The total added footprint is $N(2dr) +$
 111 $2(N+1)d + N$, covering adapters, router query/key biases, and stored scalar gates. Router tables
 112 include $N+1$ rows for the embedding source and preserve block indexing, scaling linearly with
 113 block groups and adapter rank, while all pretrained backbone weights remain frozen.

114 With $\gamma_n=0$ we have $x_n = h_{n-1}$ exactly, so the step-0 forward matches the pretrained model. The
 115 adapter up-projection is initialized nonzero rather than zero so that, once the curriculum moves $s(t)$

above zero, gradients can immediately train the routed correction path. We use a deterministic gate curriculum $\gamma_n(t) = s(t)$ with $s(0) = 0$ and $s(T) = 1$, which gradually transfers the model from the pretrained residual path to the routed correction path. At convergence every block uses the routed correction path at full strength; because the correction is bottlenecked through A_n , this is an AttnRes-conditioned residual retrofit rather than a direct replacement $x_n = r_n$. This construction is the stable bridge between two undesirable extremes: observer-only routing never affects the representation, while direct interpolation into r_n moves the frozen backbone off its pretrained input manifold.

The training objective follows from the same constraints. We minimise

$$\mathcal{L} = \mathcal{L}_{\text{CE}}(\text{full path}) + \lambda_{\text{kl}} \mathcal{L}_{\text{KL}}(\text{teacher} \parallel \text{skip path}) - \lambda_{\text{ent}} \mathbb{E}_{n,p}[H(\alpha_{\cdot \rightarrow n,p})]. \quad (4)$$

The full-path CE trains ordinary next-token prediction. The skip-branch KL randomly masks a block and distills the resulting surrogate path from the frozen teacher, making x_n meaningful both when the block executes and when ReSkip later short-circuits it. The entropy term is a light entropy-maximising prior that prevents premature one-source collapse of α . The CE term alone would not train a skipped surrogate, while teacher imitation alone would merely reproduce the base model; the combination gives an identity-preserving, skip-ready retrofit. Each loss term serves a clear purpose: prediction accuracy, stability under skipping, and diverse routing behavior. The skip branch trains robustness to occasional missing-block perturbations; deployment skips are still chosen by the calibrated ReSkip rule. In these expectations, p denotes a sequence position, reserving t for training-step schedules.

2.2 ReSkip: calibrated routing-guided block skipping

Rather than designing a new external router, we directly reuse the internal routing signal to safely skip low-impact blocks. ReSkip is a deployment policy over the phase-1 routing weights, not a new trainable module. It has three steps. First, select an eligible set $\mathcal{P}$ offline on the target distribution. For likelihood-based text evaluation we use two diagnostics:

$$I(n) = \max_{l > n} \mathbb{E}_{\text{calib},p}[\alpha_{n \rightarrow l,p}], \quad A(n) = \frac{\text{PPL}(\text{block } n \text{ removed})}{\text{PPL}(\text{full depth})}. \quad (5)$$

$I(n)$ measures how often later routers reuse block n , while $A(n)$ measures whether removing that block is safe. The position index p is averaged over calibration tokens. For VLA deployment we keep the same protocol but replace $A(n)$ with action-drift under single-block removal. Routing tells us what to skip; safety sweeps tell us what we can safely skip. Thus, $\mathcal{P}$ combines routing structure with an explicit safety sweep, rather than using small routing weights alone.

Second, calibrate a per-block threshold τ_n as the q -th empirical quantile of the deployment distribution of $w_{\text{recent}}(n) = \mathbb{E}_{\text{token}}[\alpha_{n-1 \rightarrow n}]$, the mean routing weight that block n places on its immediate predecessor. For prefill, this expectation averages over non-padding positions in the current sequence and yields one block-level execute/skip decision for that forward pass. During autoregressive decoding, the forward pass contains only the newly decoded position, so the same rule reduces to a per-step token decision; ReSkip does not route different prefill tokens through different block subsets. Third, at inference, before executing block n , apply

$$\text{skip block } n \iff w_{\text{recent}}(n) > \tau_n \text{ and } n \in \mathcal{P} \text{ and } \sum_{m < n} \mathbb{I}[\text{skipped}_m] < M_{\text{max}}, \quad (6)$$

where M_{max} caps the number of skipped blocks in one forward. The quantile q is an operating point, not a universal constant, and must be calibrated on the deployment distribution. The decision is available before block n runs because it depends only on completed outputs and the already-computed phase-1 router. By contrast, confidence-based early exits or post-block hidden-state diagnostics cannot skip the current block without first paying its cost. When the rule triggers, ReSkip sets $h_n \leftarrow x_n$, using the retrofit surrogate from Eq. 3 in place of the skipped block output. Thus skipping forwards the trained surrogate state rather than inventing a new block output.

We use both $I(n)$ and $A(n)$, because depth-attention is a routing signal, not a complete safety certificate. The eligible set should therefore combine routing structure with safety sweeps: low I identifies redundant sources, low A or low action-drift makes a candidate safe, and the calibrated w_{recent} distribution determines whether it actually fires on deployment inputs. Accordingly, ReSkip is a calibrated adaptive-execution rule, not a universal pruning criterion; modalities or deployments may need different thresholds and eligible sets.

Table 1: **340M from-scratch prior.** Accuracy scores are percentages. AttnRes full improves the matched standard-residual base on five representative benchmarks; this representative ReSkip point uses $\mathcal{P}=\{3, 5\}$, $M=2$, $q=0.85$. Appendix C reports the corresponding Pareto/latency sweep.

Benchmark	Base transformer	AttnRes full	AttnRes+ReSkip
LAMBADA acc	37.9	40.5	40.5
LAMBADA ppl ↓	24.7	20.2	20.2
HellaSwag acc_norm	44.4	46.1	46.1
PIQA acc_norm	67.8	68.9	68.9
OpenBookQA acc_norm	33.2	35.8	35.8
Latency vs. base ↓ (512 in / 4096 out)	1.0×	1.8×	1.1×

2.3 Prior: when calibrated skipping preserves predictions

To ensure skipping does not change model outputs, we derive a simple condition that bounds prediction shift relative to the model’s prediction margin. ReSkip does not assume that every block skip is harmless; it gives a sufficient condition under which a skip decision preserves the full-depth prediction. Let $S_F(x, c)$ be the full-depth score assigned to candidate answer c for input x , and let $S_R(x, c)$ be the score after applying ReSkip. We define S_F, S_R operationally in Appendix A.5. If $c^* = \arg \max_c S_F(x, c)$, define the full-depth margin and skip perturbation as

$$\Delta(x) = S_F(x, c^*) - \max_{c \neq c^*} S_F(x, c), \quad \epsilon(x) = \max_c |S_R(x, c) - S_F(x, c)|. \quad (7)$$

Whenever $2\epsilon(x) < \Delta(x)$, the prediction is unchanged:

$$S_R(x, c^*) \geq S_F(x, c^*) - \epsilon(x) > \max_{c \neq c^*} S_F(x, c) + \epsilon(x) \geq \max_{c \neq c^*} S_R(x, c). \quad (8)$$

Thus answer-level predictions are unchanged whenever the skip-induced perturbation stays below the full-depth margin: the underlying scores may move, but the winning answer does not. Intuitively, this means that if the skip-induced perturbation is small enough, the top-1 prediction remains the same. For log-likelihood quantities such as perplexity, the same $\epsilon(x)$ bounds numerical drift rather than implying exact equality. Exact equality of those quantities occurs only in the degenerate case $\epsilon(x)=0$, for example when the calibrated rule fires no skips on the evaluated inputs. In short, the model’s own prediction margin acts as a safety buffer for skipping. This is a margin-preservation condition: calibration must keep the perturbation small on the target distribution.

The skip rule in Eq. 6 is calibrated with the eligibility diagnostics in Eq. 5 and deployment quantiles to keep this perturbation small. If block n is skipped, the local hidden-state error is $\rho_n(x) = \text{Block}_n(x_n) - x_n$. For a local Lipschitz constant $L_{>n}$ of the downstream map from h_n to candidate scores, one skipped block contributes at most $L_{>n} \|\rho_n(x)\|$ to $\epsilon(x)$; multiple skipped blocks add under the triangle inequality. Offline ablation or action-drift estimates which positions have small downstream amplification, while the online w_{recent} threshold fires only when the routed input is locally close to the residual predecessor. The method is therefore a calibrated margin-preservation rule: it skips only when the selected block is globally safe enough and the current input appears locally redundant, which formalizes why our calibrated two-stage ReSkip rule preserves accuracy while skipping blocks. This separates position selection, which controls block sensitivity, from online triggering, which adapts to the current input.

3 Experiments

Backbones and retrofit recipe. We retrofit **Qwen3-VL-2B** [1] ($D=28, d=2048$) with $N=7$ blocks of $L=4$ layers each, and **Qwen3-VL-4B** ($D=36, d=2560$) with $N=9$ blocks of $L=4$ layers each. All pretrained backbone parameters are frozen. The added retrofit path contains gradient-optimized routers and residual adapters plus scheduled scalar γ gates: $\sim 7.4\text{M}$ parameters for 2B ($< 0.4\%$) and $\sim 11.8\text{M}$ for 4B ($< 0.3\%$) at adapter rank $r=256$. The SFT mix is 60% LLaVA-OneVision-Data [26], 20% UltraChat [8], 10% NuminaMath, and 10% OpenThoughts for 10k steps; the Qwen3-VL Base (LoRA) rows use LoRA adapters trained on this same mix and step budget, and are evaluated on both text and VLM benchmarks. Optimizer and schedule details are in Appendix A.1. Identity-at-init is verified before training ($\gamma=0$ reproduces base; $\max |\Delta \logits| = 0.375$ bf16, 100% argmax agreement; Appendix A.4).

Table 2: **AR-Retrofit and ReSkip on Qwen3-VL-2B/4B at the canonical $L=4$ recipe** (standard benchmark protocols for text and `lmms-eval` full splits for VLM). Base (LoRA) rows are same-data LoRA controls. Scores are reported as percentages.

Config.	Text		VLM (<code>lmms-eval</code> full split)					
	Lamb.	HSwag	MMB	MMMU	MMStar	AI2D	OCR	RWQA
2B Base (LoRA)	53.2	50.6	75.8	41.4	53.6	73.6	77.2	64.8
2B + AR-Retrofit	56.5	50.0	78.9	43.2	53.6	75.8	81.4	66.1
2B Δ	+3.3	−0.6	+3.1	+1.8	0.0	+2.2	+4.2	+1.3
2B + ReSkip	56.1	49.2	78.9	43.0	54.9	75.9	80.1	66.0
4B Base (LoRA)	57.6	56.2	83.3	49.0	62.4	81.9	81.9	71.5
4B + AR-Retrofit	66.3	55.2	85.2	52.1	63.2	82.5	82.4	71.8
4B Δ	+8.7	−1.0	+1.9	+3.1	+0.8	+0.6	+0.5	+0.3
4B + ReSkip	63.3	54.9	85.0	51.4	62.1	82.5	82.5	70.7

Benchmarks. For VLM/text evaluation we use LAMBADA [33] and HellaSwag [46] plus six `lmms-eval` VLM benchmarks: MMBench [28], MMMU [44], MMStar [5], AI2D [20], OCR-Bench [29], and RealWorldQA [43]. For VLA evaluation, we attach the OpenVLA-OFT [23] action head (L_1 regression, no chunking) to Qwen3-VL-2B/4B backbones with frozen ViT, train on the pooled `libero_all` mix from LIBERO [27], and evaluate 500 rollouts per suite (2,000 per policy).

Baselines and controls. For Qwen3-VL, Base (LoRA) denotes the same-data LoRA control without the retrofit path; Appendix D.1 adds parameter-matched text LoRA checks. MoD/LayerSkip in Tab. 3 use the same SFT mix and 10k-step budget; static, Gromov, rate-matched skipping, and ReSkip calibration are post-hoc. VLA controls use the same action head, LIBERO data, and OFT schedule; the individual policy variants are introduced where their results are reported. Unless a row explicitly names ReSkip, quality results use the full-depth retrofit forward; decode cost is included with the skip controls in Tab. 3.

3.1 VLM Retrofit Results

The first empirical question is whether the retrofit solves the installation problem without trading away the pretrained model’s capabilities. The ReSkip rows use the same retrofit checkpoints: 2B uses $\mathcal{P}=\{4\}$, $M=1$ with $q=0.95$ for text and separately calibrated VLM thresholds; 4B uses $\mathcal{P}=\{1, 2\}$, $M=2$, $q=0.85$.

The canonical retrofit answers this first question positively (Tab. 2). Full-depth AR-Retrofit improves the Base (LoRA) control on 5/6 VLM benchmarks at 2B (with MMStar unchanged in aggregate) and 6/6 at 4B, and lifts LAMBADA by +3.3pp / +8.7pp. The largest single movement appears on the MMStar math subcategory (+7.9pp at 2B, +3.9pp at 4B; full 6-cell decomposition in Tab. 10), while perception-oriented subcategories are closer to parity. HellaSwag is within ± 1.0 pp at both scales. Every block reaches $\gamma_n=1$, so the routed correction path is fully active throughout the backbone.

The second question is attribution: are these gains caused by the AttnRes pathway or simply by same-data adapter tuning? For VLM, Tab. 2 compares against Base (LoRA) controls trained on the same VLM/text SFT mix; for text, the additional parameter-matched LoRA variants in Appendix D.1 do not reproduce the LAMBADA gain. Those text-side LoRA variants produce −0.6pp over four runs, 3.9pp behind the main-table 2B retrofit. Observer-only routing learns an α signal that never feeds back into the forward, and γ -free / interpolation variants destabilise the frozen backbone before the router stabilises. The converged recipe therefore depends on the identity-preserving bridge, the real forward-path AttnRes signal, and the skip-aware surrogate loss together.

Depth-skipping controls. Tab. 3 compares Qwen3-VL-2B skip controls. MoD/LayerSkip use the same SFT mix and 10k-step budget as the retrofit; static/Gromov rows are post-hoc. Static skips block 4, Gromov- k removes k layers, MoD gates layers 12–15 at keep ratio 0.8, and LayerSkip exits at layer 24. Speeds use the same 512/4096 StaticCache setup; ReSkip uses $\mathcal{P}=\{4\}$, $M=1$, text-calibrated $q=0.95$, and target VLM calibration for image benchmarks.

Table 3: **Skip-method comparison on Qwen3-VL-2B.** Text columns use the representative 500-example skip-sweep sample; standard-protocol headline scores are in Tab. 2. MoD/LayerSkip use matched training; static/Gromov rows are post-hoc. Quality scores are percentages; decode is tokens/s for 512 input and 4096 output tokens with StaticCache.

Method	Lamb.-500	HSwag-500	AI2D	MMMU	MMStar	Decode $\uparrow$
Static block skip	37.3	53.8	58.0	33.8	33.6	73.1
Gromov-4 [12]	6.9	44.4	57.3	35.6	49.1	78.3
Gromov-8 [12]	2.0	36.5	6.8	23.9	1.9	94.6
MoD [35]	47.4	56.1	34.6	33.4	22.0	71.2
LayerSkip [10]	14.6	46.0	37.9	31.1	44.6	80.4
Base (LoRA)	53.3	55.2	73.6	41.4	53.6	69.9
AR-Retrofit full (ours)	56.4	58.7	75.8	43.2	53.6	57.2
ReSkip (ours)	56.2	57.9	75.9	43.0	54.9	67.5

Table 4: **LIBERO 4-suite success rates (%)** for Qwen3-VL-2B/4B. Entries are success rates over 500 rollouts per suite (2,000 per policy). *Base* fine-tunes the stock backbone; *Full* uses the retrofit-initialized policy without inference-time skipping; *ReSkip* applies the calibrated action-stream skip rule to the same trained policy.

Scale	Policy	Spatial	Object	Goal	Long-10	Avg	Δ vs. Base
2B	Base	94.8	99.8	97.4	91.4	95.9	—
2B	Full	97.8	99.6	98.6	92.6	97.2	+1.3
2B	ReSkip	97.6	99.2	99.0	93.6	97.4	+1.5
4B	Base	95.0	99.2	97.8	92.2	96.1	—
4B	Full	94.6	99.8	98.2	94.2	96.7	+0.7
4B	ReSkip	96.4	98.2	98.4	92.8	96.5	+0.4

237 ReSkip stays close to the full retrofit, improves MMStar, and recovers decode speed (67.5 vs. 57.2
238 tok/s; base 69.9). Faster static/pruned baselines lose much more quality, so ReSkip is a calibrated
239 deployment rule rather than a universal pruning certificate.

240 3.2 VLA Results

241 A VLA test isolates whether the retrofit *transfers* to a downstream control task. Vision tokens
242 (perceptual, early-layer [34]), language tokens (task spec.), and action tokens (motor planning) have
243 no principled reason to share effective depth; AttnRes routing gives a per-position handle on this
244 without token-level gating machinery.

245 **LIBERO quality.** The full retrofit-initialized policy answers the warm-start question positively. It
246 reaches 97.2% on 2B (+1.3pp) and 96.7% on 4B (+0.7pp). The gain concentrates on Spatial at
247 2B (+3.0pp) and Long-10 at 4B (+2.0pp), which are precisely the suites where action prediction
248 benefits from a more stable high-level backbone.

249 **ReSkip on the action stream.** We apply ReSkip at inference on the trained retrofit-initialized policies,
250 using $\mathcal{P}=\{1, 4\}$ on 2B, $\mathcal{P}=\{1, 2\}$ on 4B, $M_{\max}=2$, and action-distribution-calibrated thresholds. At
251 this operating point, ReSkip reaches 97.4% on 2B and 96.5% on 4B, preserving a positive margin
252 over the stock-backbone base while reducing executed retrofit blocks. The parameter sweep and skip
253 controls that motivate this operating point are treated as ablations in §3.3.

254 3.3 Ablation and Qualitative Results

255 The ReSkip operating point follows the rule in §2: select eligible blocks $\mathcal{P}$ offline, set thresholds
256 from a held-out quantile q of w_{recent} , and cap the number of skipped blocks by $M_{\max}$. We instantiate
257 this rule separately for text, VLM, and action distributions. The compact chosen row follows the
258 main-table operating points: VLM-2B uses $\mathcal{P}=\{4\}$, $M=1$, $q=0.95$ for text/decode, VLM-4B uses
259 $\mathcal{P}=\{1, 2\}$, $M=2$, $q=0.85$, and VLA keeps the action-calibrated $\{1, 4\}/\{1, 2\}$, $M=2$ settings. Tab. 5

Table 5: **ReSkip parameter-selection ablation, compact view.** VLM columns report LAMBADA-500 accuracy (%); VLA columns report LIBERO 4-suite average success rate (%). Parentheses report Δ against the corresponding no-skip/full reference; for VLA this is the Full row in Tab. 4 (97.2/96.7). Appendix Tab. 11 gives expanded diagnostics for the same selection logic.

Knob	Setting	VLM-2B	VLM-4B	VLA-2B	VLA-4B	Decision
<i>Eligible set and skip cap</i>						
$\mathcal{P}, M$	main per-domain setting	56.2 (−0.2)	61.6 (−4.7)	97.4 (+0.2)	96.5 (−0.2)	chosen
M	main cap or wider cap	56.2 (−0.2)	61.6 (−4.7)	97.2 (+0.0)	96.3 (−0.4)	no gain
$\mathcal{P}, M$	all, $M=4$	45.4 (−11.1)	54.2 (−12.0)	96.3 (−0.9)	89.5 (−7.2)	reject
<i>Threshold quantile</i>						
q	$M=0$	56.4	66.3	97.2	96.7	reference
q	main per-domain q	56.2 (−0.2)	61.6 (−4.7)	97.4 (+0.2)	96.5 (−0.2)	chosen
q	aggressive stress test	41.2 (−15.2)	49.2 (−17.1)	83.1 (−14.1)	94.3 (−2.4)	reject

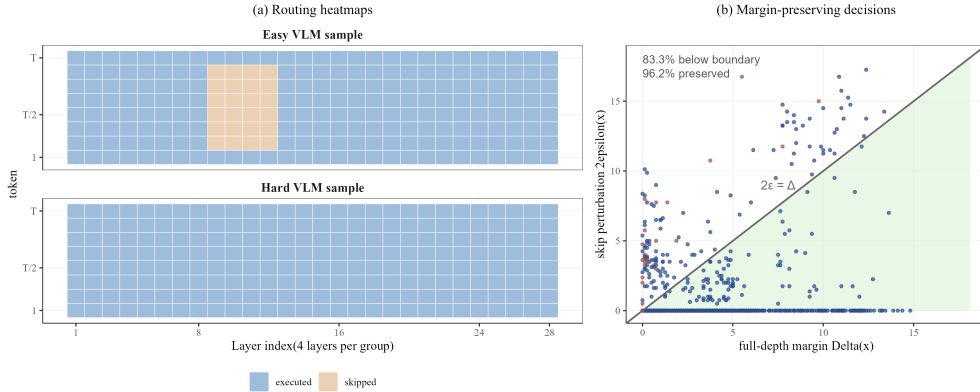


Figure 2: **ReSkip adapts depth while mostly preserving predictions.** (a) Routing heatmaps illustrate input-adaptive block skipping patterns for easy and hard samples. (b) The margin probe verifies the safe skipping condition $2\epsilon(x) < \Delta(x)$. 83.3% of tokens lie below the boundary, and 96.2% of token-level predictions are preserved under the ReSkip probe. Only a small fraction of red points above the boundary line experience a change in their top-1 prediction after skipping. Therefore predictions are ensured to remain unchanged when the condition holds.

shows the compact decision evidence; Appendix Tab. 11 provides expanded per-domain diagnostics rather than a row-by-row mirror of the compact summary.

The conclusion is consistent across modalities: choose $\mathcal{P}$ on the target model and modality, use the conservative tail of the q sweep, and keep a small skip cap ($M=1$ for the single-block VLM-2B text/decode point and $M_{\max}=2$ for the two-block settings). Wider eligible sets or lower thresholds can recover more compute, but they move off the quality-preserving part of the Pareto. The only non-average VLA diagnostic we keep outside the table is the cross-scale $\mathcal{P}$ transfer failure: applying the 2B set $\{1, 4\}$ to 4B drops LIBERO-Spatial to 12.4% (Appendix E). Orthogonal recipe evidence consists of the block-partition sweep in Tab. 9, the data-mix screening diagnostics in Appendix B.5, the retrofit recipe ablation in Appendix B.3, and the VLA calibration diagnostics in Appendix E.

Qualitative visualizations are in Fig. 2 and Fig. 3.

Figure 2: token-level adaptive-depth diagnostic on MMBench. This figure illustrates input-dependent routing and margin-preservation in ReSkip. Panel (a) shows an easy sample with a larger full-depth margin repeatedly triggering skip blocks, while a hard sample with a smaller margin executes the full path, highlighting input-conditional depth allocation rather than static pruning. Panel (b) connects this behavior to Eq. 8, where $\Delta(x)$ is the top-1 minus top-2 logit margin and $\epsilon(x)$ the maximum logit shift after skipping. In this probe, 83.3% of decisions satisfy $2\epsilon(x) \leq \Delta(x)$, and 96.2% preserve the top-token prediction. The key point: many skips occur where the margin absorbs the perturbation.

Figure 3: routing importance vs. removal cost on fineweb-edu-100BT. Using the 340M from-scratch block-AttnRes transformer, learned routing weights provide a pre-execution signal but do not guarantee safety. Fig. 3 shows $I(n)$ (routing importance) and $A(n)$ (removal cost) are complementary.

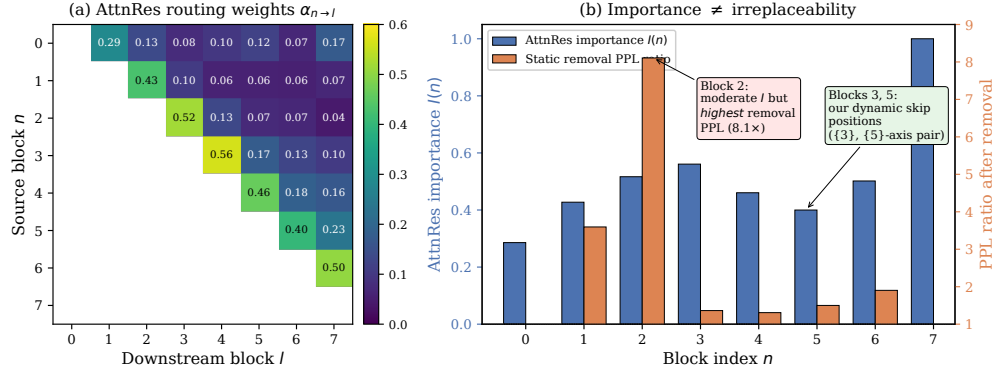


Figure 3: **AttnRes routing weights do not predict ablation impact (340M).** (a) Learned block-level $\alpha_{n \rightarrow l}$. (b) $I(n)$ (blue, left axis) and $A(n)$ (orange, right axis). ReSkip uses routing for input-dependent decisions, but eligible blocks still need ablation-based safety selection.

This motivates the two-stage skip rule: routing supplies the input-dependent trigger, and an ablation or action-drift sweep ensures the block is safe to skip. The representative ReSkip point in Table 1 is thus a calibrated operating point, not a universal pruning certificate.

4 Related Work

For brevity, we discuss only the most closely related papers. Test-time compute scaling extends token traces but keeps depth per token fixed, as in o1, DeepSeek-R1, chain-of-thought prompting, and compute scheduling [32, 13, 42, 37]. Depth-adaptive work includes Stochastic Depth [16], ACT/Universal Transformers [11, 6], CALM [36], Mixture-of-Depths [35], LayerSkip [10], and static pruning [12, 31]. These systems typically rely on halting heads, exits, token routers, speculation, or pruning rules. AR-Retrofit instead installs AttnRes-style routing inside a pretrained backbone, so the forward computation itself yields the depth signal.

Our motivation also echoes recurrent/effective-depth views: cognitive and visual accounts link harder inputs to deeper recurrent computation [18, 24, 19, 7, 21, 38], finite recurrent steps can be unrolled into feed-forward depth [41], and cortical/dendritic accounts motivate internally generated routing [2, 4, 25]. We use this as design intuition, not a biological claim.

Residual architectures such as DenseNet [17], Highway Networks [39], and AttnRes [40] change information flow, while adapters and LoRA [14, 15] add small local updates to frozen models. AR-Retrofit differs from scratch-trained AttnRes and conventional PEFT by retrofitting pretrained transformers with routed aggregation that also exposes skip-ready depth signals. RT-2, Octo, OpenVLA, and Pi0 define the VLA setting [48, 30, 22, 3]; action chunking is orthogonal [47]. We add adaptive depth inside the VLM backbone.

5 Conclusion

We introduced AR-Retrofit, a post-hoc way to add AttnRes-style depth aggregation to frozen pre-trained decoders, and ReSkip, a calibrated rule that turns the learned phase-1 routing weights into adaptive block execution. Across Qwen3-VL-2B/4B, the retrofit improves VLM and language benchmarks with less than 0.4% new parameters, transfers as a stronger warm start for LIBERO VLA policies, and gives ReSkip a usable signal for recovering inference cost. The main caveat is calibration: skip sets and thresholds must be chosen for the target model and modality. The retrofit recipe also depends on the SFT mix and is evaluated here within the Qwen3-VL family; extending it to other VLM and text-only backbones is future work. By lowering inference cost, the method can make capable models easier to deploy, which is useful for accessibility but can also lower barriers to misuse and should inherit the safeguards of the underlying backbone. Overall, these results show that already trained decoders can gain a complementary depth-axis computation mechanism without full retraining.

References

- [1] Shuai Bai, Yuxuan Cai, Ruizhe Chen, Keqin Chen, Xionghui Chen, Zesen Cheng, Lianghao Deng, Wei Ding, Chang Gao, Chunjiang Ge, et al. Qwen3-vl technical report. *arXiv preprint arXiv:2511.21631*, 2025.
- [2] Andre M Bastos, W Martin Usrey, Rick A Adams, George R Mangun, Pascal Fries, and Karl J Friston. Canonical microcircuits for predictive coding. *Neuron*, 76(4):695–711, 2012.
- [3] Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- [4] Timothy J Buschman and Earl K Miller. Top-down versus bottom-up control of attention in the prefrontal and posterior parietal cortices. *science*, 315(5820):1860–1862, 2007.
- [5] Lin Chen, Jinsong Li, Xiaoyi Dong, Pan Zhang, Yuhang Zang, Zehui Chen, Haodong Duan, Jiaqi Wang, Yu Qiao, Dahua Lin, et al. Are we on the right way for evaluating large vision-language models? *Advances in Neural Information Processing Systems*, 37:27056–27087, 2024.
- [6] Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Lukasz Kaiser. Universal transformers. In *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*. OpenReview.net, 2019. URL <https://openreview.net/forum?id=HyzdRiR9Y7>.
- [7] James J DiCarlo, Davide Zoccolan, and Nicole C Rust. How does the brain solve visual object recognition? *Neuron*, 73(3):415–434, 2012.
- [8] Ning Ding, Yulin Chen, Bokai Xu, Yujia Qin, Shengding Hu, Zhiyuan Liu, Maosong Sun, and Bowen Zhou. Enhancing chat language models by scaling high-quality instructional conversations. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 3029–3051, 2023.
- [9] Maha Elbayad, Jiatao Gu, Edouard Grave, and Michael Auli. Depth-adaptive transformer. In *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*. OpenReview.net, 2020. URL <https://openreview.net/forum?id=SJg7KhVKPH>.
- [10] Mostafa Elhoushi, Akshat Shrivastava, Diana Liskovich, Basil Hosmer, Bram Wasti, Liangzhen Lai, Anas Mahmoud, Bilge Acun, Saurabh Agarwal, Ahmed Roman, et al. Layerskip: Enabling early exit inference and self-speculative decoding. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 12622–12642, 2024.
- [11] Alex Graves. Adaptive computation time for recurrent neural networks. *arXiv preprint arXiv:1603.08983*, 2016.
- [12] Andrey Gromov, Kushal Tirumala, Hassan Shapourian, Paolo Glorioso, and Daniel A Roberts. The unreasonable ineffectiveness of the deeper layers. *arXiv preprint arXiv:2403.17887*, 2024.
- [13] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.
- [14] Neil Houlsby, Andrei Giurgiu, Stanislaw Jastrzebski, Bruna Morrone, Quentin De Laroussilhe, Andrea Gesmundo, Mona Attariyan, and Sylvain Gelly. Parameter-efficient transfer learning for nlp. In *International conference on machine learning*, pages 2790–2799. PMLR, 2019.
- [15] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Liang Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models. *Iclr*, 1(2):3, 2022.

- [16] Gao Huang, Yu Sun, Zhuang Liu, Daniel Sedra, and Kilian Q Weinberger. Deep networks with stochastic depth. In *European conference on computer vision*, pages 646–661. Springer, 2016.
- [17] Gao Huang, Zhuang Liu, Laurens Van Der Maaten, and Kilian Q Weinberger. Densely connected convolutional networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 4700–4708, 2017.
- [18] Daniel Kahneman. *Thinking, fast and slow*. macmillan, 2011.
- [19] Kohitij Kar, Jonas Kubilius, Kailyn Schmidt, Elias B Issa, and James J DiCarlo. Evidence that recurrent circuits are critical to the ventral stream’s execution of core object recognition behavior. *Nature neuroscience*, 22(6):974–983, 2019.
- [20] Aniruddha Kembhavi, Mike Salvato, Eric Kolve, Minjoon Seo, Hannaneh Hajishirzi, and Ali Farhadi. A diagram is worth a dozen images. In *European conference on computer vision*, pages 235–251. Springer, 2016.
- [21] Tim C Kietzmann, Courtney J Spoerer, Lynn KA Sörensen, Radoslaw M Cichy, Olaf Hauk, and Nikolaus Kriegeskorte. Recurrence is required to capture the representational dynamics of the human visual system. *Proceedings of the National Academy of Sciences*, 116(43):21854–21863, 2019.
- [22] Moo Jin Kim, Karl Pertsch, Siddharth Karamcheti, Ted Xiao, Ashwin Balakrishna, Suraj Nair, Rafael Rafailov, Ethan Foster, Grace Lam, Pannag Sanketi, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- [23] Moo Jin Kim, Chelsea Finn, and Percy Liang. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025.
- [24] Victor AF Lamme and Pieter R Roelfsema. The distinct modes of vision offered by feedforward and recurrent processing. *Trends in neurosciences*, 23(11):571–579, 2000.
- [25] Matthew Larkum. A cellular mechanism for cortical associations: an organizing principle for the cerebral cortex. *Trends in neurosciences*, 36(3):141–151, 2013.
- [26] Bo Li, Yuanhan Zhang, Dong Guo, Renrui Zhang, Feng Li, Hao Zhang, Kaichen Zhang, Peiyuan Zhang, Yanwei Li, Ziwei Liu, et al. Llava-onevision: Easy visual task transfer. *arXiv preprint arXiv:2408.03326*, 2024.
- [27] Bo Liu, Yifeng Zhu, Chongkai Gao, Yihao Feng, Qiang Liu, Yuke Zhu, and Peter Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. *Advances in Neural Information Processing Systems*, 36:44776–44791, 2023.
- [28] Yuan Liu, Haodong Duan, Yuanhan Zhang, Bo Li, Songyang Zhang, Wangbo Zhao, Yike Yuan, Jiaqi Wang, Conghui He, Ziwei Liu, et al. Mmbench: Is your multi-modal model an all-around player? In *European conference on computer vision*, pages 216–233. Springer, 2024.
- [29] Yuliang Liu, Zhang Li, Mingxin Huang, Biao Yang, Wenwen Yu, Chunyuan Li, Xu-Cheng Yin, Cheng-Lin Liu, Lianwen Jin, and Xiang Bai. Ocrbench: on the hidden mystery of ocr in large multimodal models. *Science China Information Sciences*, 67(12):220102, 2024.
- [30] Oier Mees, Dibya Ghosh, Karl Pertsch, Kevin Black, Homer Rich Walke, Sudeep Dasari, Joey Hejna, Tobias Kreiman, Charles Xu, Jianlan Luo, et al. Octo: An open-source generalist robot policy. In *First Workshop on Vision-Language Models for Navigation and Manipulation at ICRA 2024*, 2024.
- [31] Xin Men, Mingyu Xu, Qingyu Zhang, Qianhao Yuan, Bingning Wang, Hongyu Lin, Yaojie Lu, Xianpei Han, and Weipeng Chen. Shortgpt: Layers in large language models are more redundant than you expect. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 20192–20204, 2025.
- [32] OpenAI. Learning to reason with LLMs. <https://openai.com/index/learning-to-reason-with-llms/>, 2024.

- [33] Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Ngoc-Quan Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernández. The lambada dataset: Word prediction requiring a broad discourse context. In *Proceedings of the 54th annual meeting of the association for computational linguistics (volume 1: Long papers)*, pages 1525–1534, 2016.
- [34] Maithra Raghu, Thomas Unterthiner, Simon Kornblith, Chiyuan Zhang, and Alexey Dosovitskiy. Do vision transformers see like convolutional neural networks? *Advances in neural information processing systems*, 34:12116–12128, 2021.
- [35] David Raposo, Sam Ritter, Blake Richards, Timothy Lillicrap, Peter Conway Humphreys, and Adam Santoro. Mixture-of-depths: Dynamically allocating compute in transformer-based language models. *arXiv preprint arXiv:2404.02258*, 2024.
- [36] Tal Schuster, Adam Fisch, Jai Gupta, Mostafa Dehghani, Dara Bahri, Vinh Tran, Yi Tay, and Donald Metzler. Confident adaptive language modeling. *Advances in Neural Information Processing Systems*, 35:17456–17472, 2022.
- [37] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling llm test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024.
- [38] Courtney J Spoerer, Tim C Kietzmann, Johannes Mehrer, Ian Charest, and Nikolaus Kriegeskorte. Recurrent neural networks can explain flexible trading of speed and accuracy in biological vision. *PLoS computational biology*, 16(10):e1008215, 2020.
- [39] Rupesh Kumar Srivastava, Klaus Greff, and Jürgen Schmidhuber. Highway networks. *arXiv preprint arXiv:1505.00387*, 2015.
- [40] Kimi Team, Guangyu Chen, Yu Zhang, Jianlin Su, Weixin Xu, Siyuan Pan, Yaoyu Wang, Yucheng Wang, Guanduo Chen, Bohong Yin, et al. Attention residuals. *arXiv preprint arXiv:2603.15031*, 2026.
- [41] Ruben S van Bergen and Nikolaus Kriegeskorte. Going in circles is the way forward: the role of recurrence in visual inference. *Current Opinion in Neurobiology*, 65:176–193, 2020.
- [42] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
- [43] xAI. Realworldqa: A benchmark for real-world spatial understanding. <https://huggingface.co/datasets/xai-org/RealworldQA>, 2024. Accessed: 2025-04-26.
- [44] Xiang Yue, Yuansheng Ni, Kai Zhang, Tianyu Zheng, Ruqi Liu, Ge Zhang, Samuel Stevens, Dongfu Jiang, Weiming Ren, Yuxuan Sun, et al. Mmmu: A massive multi-discipline multimodal understanding and reasoning benchmark for expert agi. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 9556–9567, 2024.
- [45] Eric Zelikman, Georges Harik, Yijia Shao, Varuna Jayasiri, Nick Haber, and Noah D Goodman. Quiet-star: Language models can teach themselves to think before speaking. *arXiv preprint arXiv:2403.09629*, 2024.
- [46] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. Hellaswag: Can a machine really finish your sentence? In *Proceedings of the 57th annual meeting of the association for computational linguistics*, pages 4791–4800, 2019.
- [47] Tony Z. Zhao, Vikash Kumar, Sergey Levine, and Chelsea Finn. Learning fine-grained bimanual manipulation with low-cost hardware. In Kostas E. Bekris, Kris Hauser, Sylvia L. Herbert, and Jingjin Yu, editors, *Robotics: Science and Systems XIX, Daegu, Republic of Korea, July 10-14, 2023*, 2023. doi: 10.15607/RSS.2023.XIX.016. URL <https://doi.org/10.15607/RSS.2023.XIX.016>.
- [48] Brianna Zitkovich, Tianhe Yu, Sichun Xu, Peng Xu, Ted Xiao, Fei Xia, Jialin Wu, Paul Wohlhart, Stefan Welker, Ayzaan Wahid, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*, pages 2165–2183. PMLR, 2023.

A Implementation details

A.1 Retrofit hyperparameters

AdamW with $\beta=(0.9, 0.95)$, weight decay 0, gradient clip 1.0, bfloat16. Learning rate 10^{-3} for gradient-optimized retrofit params (routers and adapters; γ is scheduled as below), cosine schedule with 100-step warmup. Sequence length 1536 (training), 2048 (evaluation). Loss weights $\lambda_{kl}=1$, entropy weight 0.02, KD temperature 1. The entropy term is subtracted from the loss, so it maximises routing entropy early in training rather than minimising it. Skip-branch sampling: one eligible block chosen uniformly at random per step. Adapter bottleneck rank $r=256$ (canonical); position-bias on router keys enabled; router temperature fixed at 1. γ -curriculum: $0 \rightarrow 1$ linearly over the first 30% of total steps (2B 5k/10k, 4B 5k), with ramp-fraction 0.5 on $4B \times 10k$ to stabilise a late-stage $\gamma=1$ transition divergence (Appendix A.6). Main 10k retrofit runs use $4 \times H100$ GPUs in bfloat16: about 15 minutes for Qwen3-VL-2B and 33 minutes for Qwen3-VL-4B (1.0/2.2 H100-hours); ablation and screening runs reuse this setup unless noted.

A.2 Existing assets and licenses

We use existing assets under their original licenses and terms, and do not redistribute or repackage them. Qwen3-VL-2B/4B, LLaVA-OneVision-Data, NuminaMath-CoT, and OpenThoughts3 are Apache-2.0; UltraChat is MIT; FineWeb-Edu is ODC-By 1.0 and subject to Common Crawl terms. LIBERO code/data are MIT/CC BY 4.0, and OpenVLA-OFT is MIT. Evaluation uses the cited benchmark splits and public protocols; lm-evaluation-harness is MIT, while lmms-eval keeps MIT for inherited harness code and Apache-2.0 for added multimodal components. Any public code release after de-anonymization will retain required notices.

A.3 Block granularity

For the final recipe we use $L=4$ layers per block on both scales: Qwen3-VL-2B has $N=7$ blocks and Qwen3-VL-4B has $N=9$ blocks. The block-partition sweep in Tab. 9 shows that $L=4$ preserves the accuracy plateau while reducing router calls. Each block receives one router, one residual adapter ($r=256$ default), and a scalar γ gate.

A.4 Identity-at-init details

With $\gamma_n=0$ and the adapter up-projection initialised at $\mathcal{N}(0, 0.02^2)$, the retrofit forward is arithmetically identical to the unmodified pretrained checkpoint (both produce $x_n = h_{n-1}$, which is passed into the unmodified block). We verified end-to-end: at $n=500$ on LAMBADA the retrofit and that unmodified checkpoint agree to within bfloat16 evaluation noise (acc 0.530 vs. 0.532; ppl 5.572 vs. 5.547); on MMMU and MMStar they are bit-equal at the answer-choice level. The smoke test at the token level shows $\max |\Delta \logits| = 0.375$ (bfloat16 SDPA noise) and 100% argmax agreement on our calibration prompts.

A.5 Skip-path correctness verification

We verified that the cached skip path is numerically consistent with the cacheless path. Across representative skip configurations on Qwen3-VL-2B, the last-position argmax matches and the maximum logit delta remains in the same range as the bfloat16 SDPA jitter observed on the unmodified pretrained checkpoint. Thus the measured differences are numerical-evaluation noise rather than a bug in the skip path.

Score computation for the margin probe. For discrete LLM/VLM predictions, the candidate c denotes either a next token or an answer string. Given an answer string $c = (y_1, \dots, y_T)$, we run the same input with the full-depth forward and with ReSkip enabled, obtaining token distributions p_F and p_R . We score c by average log-likelihood:

$$S_F(x, c) = \frac{1}{T} \sum_{t=1}^T \log p_F(y_t \mid x, y_{<t}), \quad S_R(x, c) = \frac{1}{T} \sum_{t=1}^T \log p_R(y_t \mid x, y_{<t}).$$

For token-level probes, this reduces to scoring next-token candidates, where we use the corresponding logits as S_F and S_R . The margin condition in Sec. 2.3 applies to these discrete-score settings. For VLA, the policy output is continuous rather than a finite candidate set, so we do not use a discrete margin; instead, calibration measures the action-output drift between the full and ReSkip forwards, e.g.,

$$D_a(x) = \frac{1}{d_a} \|a_R(x) - a_F(x)\|_2^2.$$

A.6 $\gamma=1$ transition stability

Under the v3 mix with ramp-fraction 0.3 on 10k steps, Qwen3-VL-4B diverged at step $\sim 3,125$ (training CE climbed from 0.9 to 6.5+). The same configuration at 5k steps, and at 10k steps with the v3-VL-only mix (same ramp-fraction but reduced reasoning-gradient density), converged stably. Extending the ramp to fraction 0.5 (ramp ends at step 5,000 rather than step 3,000) recovered convergence on $4B \times 10k$ with no measurable effect on final quality: $4B \times 10k$ at ramp 0.5 is within 1pp of $4B \times 5k$ at ramp 0.3 on every VLM benchmark, so the ramp-fraction difference does not confound the between-cell comparison in Tab. 9. Working rule: the $\gamma=1$ transition is stable whenever the schedule reaches $\gamma=1$ at step $\geq 5,000$, across every scale and partition we tested.

B Retrofit ablations and inference machinery

B.1 Rejected retrofit controls

We tried three simpler controls before the final γ -gated residual injection: observer-only routing, direct interpolation toward the routed state, and direct AttnRes replacement with informed pseudo-query initialization. Observer-only routing produced a signal that did not affect the forward path; direct interpolation moved the frozen backbone away from its pretrained input manifold; and direct replacement was unstable under the short retrofit budget. These failures motivate the identity-preserving bridge used in §2.1.

B.2 Adapter-rank ablation

Wall-clock latency is essentially unchanged across ranks ($\pm 2\%$ at all measured sequence lengths): the adapter’s $2048 \rightarrow r \rightarrow 2048$ bottleneck is a small contributor compared to the router stack/softmax/einsum cost. Rank therefore affects quality but not speed. The final $r=256$ choice was selected from the $\gamma \rightarrow 1$ ablation of Appendix B.3; at equal step budgets lower ranks leave more of the MMBench drop on the table without compensating latency savings.

B.3 $\gamma \rightarrow 1$ recipe-selection ablation

Before the final v3 $L=4$ headline run, we swept 8 recipe-selection variants along four axes: adapter rank $r \in \{32, 64, 128, 256\}$, training steps $\in \{5k, 10k, 20k\}$, LLaVA-Instruct fraction of the mix $\in \{50\%, 80\%\}$, and γ ramp-fraction $\in \{0.3, 0.7\}$. All other hyperparameters (loss, optimizer, schedule family, seed) are held constant. A separate γ -free control is run for the same number of steps with $\gamma_n \equiv 0$ trainable around its zero-init (Appendix Table 6). Headline scores use the v3 $L=4$ rows in Tab. 2; this table documents the selection-stage choice of the γ bridge and $r=256$ adapter.

The sweep identifies two practical constraints. First, longer training after the $\gamma=1$ transition can over-specialise the router and hurt VLM calibration. Second, lower-rank adapters reduce parameters but do not recover the selected quality/latency tradeoff. The γ -free row is retained as an adapter-dominated control: it can improve some scores, but it does not install the same full-strength routed correction path used by ReSkip.

B.4 Calibration set (dynamic skip)

Text calibration uses 32 held-out LAMBADA prefixes truncated to 512 tokens. For each eligible block we collect $w_{\text{recent}}(n)$ and set τ_n to the empirical q -quantile. VLA deployment uses the same computation on action-rollout records rather than text prefixes, because the action stream has a different routing distribution.

Table 6: **Recipe-selection retrofit ablation** on Qwen3-VL-2B. γ -free: γ_n trainable around 0, adapter-dominated (opposite structural extreme to the selected $\gamma \rightarrow 1$ bridge). All others: γ -curriculum $0 \rightarrow 1$ over the first 30% of steps. “n=300 MMBench noise floor ± 1 question” corresponds to ± 0.33 pp.

Name	γ sched	rank	steps	VLM%	ramp	LAMBADA	HellaSwag	MMBench
Base Qwen3-VL-2B	—	—	—	—	—	0.532	0.506	0.727
γ -free (A)	$\gamma \approx 0$ trainable	128	10k	50	—	0.576	0.512	0.720
Selection-stage winner	$0 \rightarrow 1$ fast	256	5k	50	0.3	0.576	0.522	0.710
+ low rank	$0 \rightarrow 1$ fast	64	10k	50	0.3	0.570	0.530	0.697
+ very low rank	$0 \rightarrow 1$ fast	32	10k	50	0.3	0.568	0.526	0.683
+ low rank, VLM-heavy	$0 \rightarrow 1$ fast	64	10k	80	0.3	0.560	0.524	0.660
+ mid rank, VLM-heavy	$0 \rightarrow 1$ fast	128	10k	80	0.3	0.564	0.520	0.653
+ VLM-heavy	$0 \rightarrow 1$ fast	256	10k	80	0.3	0.560	0.520	0.617
+ longer, 10k	$0 \rightarrow 1$ fast	256	10k	50	0.3	0.586	—	0.587
+ slow ramp	$0 \rightarrow 1$ slow	256	10k	50	0.7	0.568	0.520	0.517
+ longer, 20k	$0 \rightarrow 1$ fast	256	20k	50	0.3	0.568	0.520	0.513

549 B.5 Data-mix screening diagnostics (v1 $\rightarrow$ v2 $\rightarrow$ v3)

550 Table 7 reports the non-canonical data-mix screening runs that motivated the final v3 recipe; canonical
551 v3 quality is reported once in the main results (Tab. 2) and in the block-partition sweep (Tab. 9).
552 The screening rows show how the mix landscape partitions: v1 (narrow VL: UltraChat + LLaVA-
553 Instruct-VSFT) is the initial canonical and preserves base within noise at 5k steps but *collapses* when
554 trained longer; v2 (aggressive math-CoT: 40% NuminaMath/OpenThoughts/OpenMath2 +30% VL)
555 recovers MMStar reasoning subtasks on 2B but *crashes AI2D* by 45pp (on 2B) and the global VL
556 on 4B. The selected v3 mix was introduced to avoid these two failure modes, not to create a second
557 source of headline v3 numbers in this appendix.

Table 7: **Data-mix screening diagnostics.** Non-canonical screening runs use the same $\gamma \rightarrow 1$ retrofit recipe ($r=256$, $L=2$, ramp-fraction per-scale) while varying only the data mix and step count. v1 at 5k is preserved, but at 10k it collapses on 4B (AI2D -23 pp). v2 crashes diagram-reasoning on 2B despite raising MMStar math subtasks. Canonical v3 results are not repeated here; they are reported in Tab. 2 and Tab. 9. All numbers from 1mms-eval full splits; MMBench is a percentage, while the other benchmark columns are fractions.

Scale	Mix (steps)	AI2D	MMBench (%)	MMMU	MMStar	OCRBench	RealWorldQA
2B	base	0.736	75.77	0.414	0.536	0.772	0.648
2B	v1 (5k)	0.684	72.85	0.406	0.386	0.795	0.447
2B	v1 (10k)	0.677	73.71	0.404	0.471	0.801	0.642
2B	v2 (5k)	0.283	73.80	0.421	0.422	0.806	0.512
4B	base	0.819	83.33	0.490	0.624	0.819	0.715
4B	v1 (5k)	0.810	83.76	0.510	0.579	0.812	0.707
4B	v1 (10k)	0.580	81.79	0.432	0.437	0.812	0.689
4B	v2 (5k)	0.603	81.87	0.477	0.333	0.808	0.686

558 The table crystallises three working rules for the retrofit’s data mix. (i) *Anchor VL at $\geq 60\%$.*
559 *Dropping VL below 30% (v2) catastrophically breaks diagram reasoning.* Holding VL at 50% (v1)
560 *works at 5k but not at 10k.* The canonical v3 results in Tabs. 2 and 9 then show that holding VL at
561 *60% with a rich OneVision anchor works at 10k at both scales.* (ii) *Cap math-CoT text at $\leq 20\%$.*
562 *v2’s 40% math-CoT share on 2B dropped AI2D to 0.283—the retrofit’s router learnt to route away*
563 *from vision-heavy paths when the text side of the mix presented a strong symbolic-reasoning gradient.*
564 *The final v3 mix keeps math-CoT at 20%; its canonical AI2D results are reported with the main*
565 *recipe rather than duplicated in this screening table.* (iii) *More steps does not rescue a narrow mix.*
566 *v1 at 10k crashes AI2D by 23pp on 4B while barely moving 2B, disproving the hypothesis that the*
567 *v1 $\rightarrow$ v2 transition was just under-training.* The mix quality is binding.

568 B.6 Two-phase forward with dynamic skip (algorithm)

Algorithm 1 Two-phase AttnRes forward with dynamic ReSkip.

```

1: Input: embeddings  $\mathbf{h}_0$ ; block groups  $\text{Block}_1, \dots, \text{Block}_N$ ; retrofit routers/adapters/gates; eligi-
   ble  $\mathcal{P}$ ; thresholds  $\{\tau_n\}$ ; max skips  $M_{\max}$ .
2: Initialise completed states  $\mathcal{H} \leftarrow [\mathbf{h}_0]$ ; skip counter  $k \leftarrow 0$ .
3: for  $n = 1$  to  $N$  do
4:   Compute  $\alpha_{\cdot \rightarrow n}$ , routed state  $\mathbf{r}_n$ , and merged input  $\mathbf{x}_n$  from  $\mathcal{H}$  (phase I).
5:    $w_{\text{recent}}(n) \leftarrow \mathbb{E}_{\text{token}}[\alpha_{n-1 \rightarrow n}]$ .
6:   Prefill averages over non-padding sequence positions; single-token decoding makes the same
       line a per-step decision.
7:   if  $n \in \mathcal{P}$  and  $k < M_{\max}$  and  $w_{\text{recent}}(n) > \tau_n$  then
8:     Skip: set  $\mathbf{h}_n \leftarrow \mathbf{x}_n$ ;  $k \leftarrow k + 1$ .
9:   else
10:    Execute:  $\mathbf{h}_n \leftarrow \text{Block}_n(\mathbf{x}_n)$ .
11:  end if
12:  Append  $\mathbf{h}_n$  to  $\mathcal{H}$ .
13: end for
14: Output: final hidden state  $\mathbf{h}_N$ .

```

569 C 340M from-scratch validation

570 The 340M from-scratch experiment is used only as a mechanism check: AttnRes improves the
571 standard-residual baseline, and calibrated ReSkip recovers part of the added cost without requiring an
572 auxiliary router. Figure 4 shows the Pareto/latency sweep for the representative setting in Table 1.

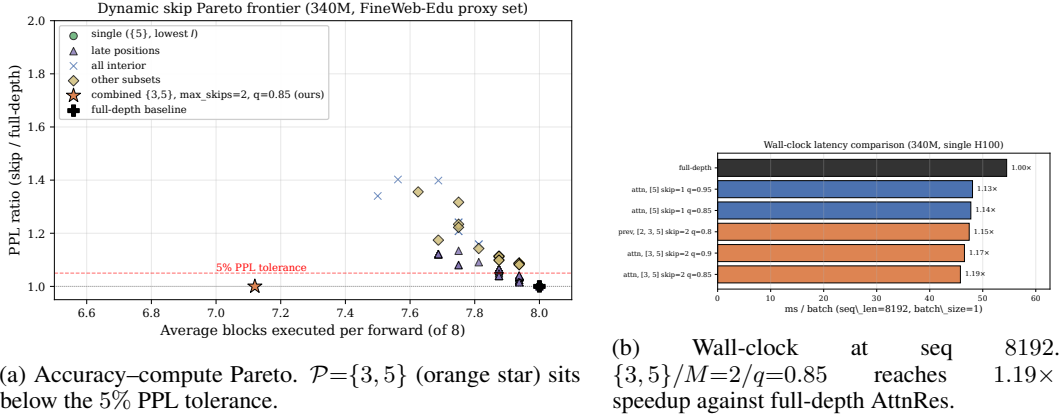


Figure 4: ReSkip on the 340M FineWeb-Edu long-context sweep.

573 D Retrofit Pareto and breakdowns

574 D.1 LoRA baselines

575 For VLM attribution, the main-table Base (LoRA) rows are same-data LoRA controls evaluated
576 on the full VLM suite. The four variants below are additional parameter-matched text-side checks
577 trained under the same SFT budget. LoRA $r=32$ on q, v : LAMBADA acc 0.540 / 0.516 (two seeds),
578 HellaSwag 0.510 / 0.524. LoRA $r=16$ on q, k, v, o : LAMBADA 0.534, HellaSwag 0.492. LoRA
579 $r=8$ on MLP: LAMBADA 0.514, HellaSwag 0.510. Mean LAMBADA across the four controls is
580 0.526, 0.6pp below the 2B Base (LoRA) row (0.532) and 3.9pp below the main-table 2B retrofit
581 (0.565). Mean HellaSwag is 0.509, within the base-level noise band, so within these additional
582 text-side LoRA checks, the relevant attribution test is LAMBADA: the gain comes from the AttnRes
583 routing structure rather than from a parameter-matched LoRA update on the same SFT data.

584 D.2 ReSkip on the canonical retrofit: text Pareto and cross-modality consistency

585 On the canonical $L=4$ v3 10k retrofit, the main-table text/decode ReSkip point uses
 586 $\mathcal{P}=\{4\}$, $M=1$, $q=0.95$. As a separate language-modality cross-check of the LIBERO Pareto
 587 (Tab. 11), we also evaluate the VLA-style $\mathcal{P}=\{1, 4\}$, $M_{\max}=2$ setting on LAMBADA-500. ReSkip
 588 produces the same shape as on action: a small drop at conservative settings, collapsing once q moves
 589 below the action distribution’s mean.

Table 8: **Cross-modality ReSkip consistency.** VLA-style text probe: LAMBADA-500 on the canonical $L=4$ v3 retrofit, $\mathcal{P}=\{1, 4\}$, $M=2$, τ calibrated on 32 held-out LAMBADA prefixes at quantile q . Same operating-point structure as the LIBERO 4-suite Pareto: small drop above the calibration distribution mean ($q \geq 0.85$); rapid collapse below, where the threshold instructs the router to skip blocks it is normally executing.

590

q	LAMBADA acc	LAMBADA ppl	Δ acc vs no-skip	avg. skips / forward
no-skip ($M=0$)	0.5700	4.526	—	$0.00 / \leq 7$
0.85	0.5600	5.258	−1.0	$0.19 / 2$
0.50	0.4120	12.550	−15.8	$1.06 / 2$
0.30	0.3900	14.005	−18.0	$1.17 / 2$

Table 9: Block partition sweep (v3 recipe, $r=256$, γ -curriculum). L is layers-per-block, $N=L_{\text{total}}/L$. Per-layer $L=1$ underperforms by 2–9pp; the plateau $L \in \{2, 4, 7\}$ on 2B reproduces Team et al. [40]’s $S \in \{2, 4, 8\}$. We recommend $L=4$ on both scales because it preserves quality while reducing router calls. MMBench is a percentage; other score columns are fractions.

Scale	L	N	LAMBADA	Δ Lamb.	HellaSwag	MMBench (%)	MMMU	MMStar	AI2D	OCRBench	RWQA
2B Base	—	—	0.532	—	0.506	75.77	0.414	0.536	0.736	0.772	0.648
(LoRA)	—	—	0.532	—	0.506	75.77	0.414	0.536	0.736	0.772	0.648
2B retrofit	1	28	0.5645	+3.3	0.490	76.20	0.388	0.499	0.743	0.809	0.652
2B retrofit	2	14	0.5755	+4.4	0.494	77.23	0.426	0.532	0.748	0.803	0.663
2B (rec.)	4	7	0.5650	+3.3	0.500	78.87	0.432	0.536	0.758	0.814	0.661
2B retrofit	7	4	0.5155	−1.7	0.492	77.49	0.427	0.530	0.756	0.808	0.656
4B Base	—	—	0.576	—	0.562	83.33	0.490	0.624	0.819	0.819	0.715
(LoRA)	—	—	0.576	—	0.562	83.33	0.490	0.624	0.819	0.819	0.715
4B retrofit	1	36	0.5575	−1.9	0.523	83.33	0.497	0.538	0.783	0.768	0.694
4B retrofit	2	18	—	—	—	84.28	0.523	0.587	0.816	0.813	0.708
4B (rec.)	4	9	0.6625	+8.7	0.552	85.22	0.521	0.632	0.825	0.824	0.718
4B retrofit	6	6	0.6540	+7.8	0.554	84.79	0.531	0.623	0.817	0.824	0.715

591

Table 10: MMStar subcategory breakdown for the v3 retrofit. The largest movement is the math cell (+7.9pp on 2B; +3.9pp on 4B), while logical and sci&tech are mixed. Perception-heavy cells (coarse / fine / instance) stay closer to parity, consistent with larger gains on math than on perception-heavy cells.

Configuration	math	logical	sci&tech	coarse	fine	instance
Qwen3-VL-2B Base (LoRA)	0.413	0.429	0.408	0.714	0.520	0.710
+ AR-Retrofit v3 (2B, $L=4$)	0.492	0.432	0.353	0.734	0.505	0.683
Δ vs. 2B Base (LoRA)	+7.9	+0.3	−5.5	+2.0	−1.5	−2.7
Qwen3-VL-4B Base (LoRA)	0.549	0.626	0.465	0.788	0.611	0.705
+ AR-Retrofit v3 (4B, $L=4$)	0.588	0.602	0.467	0.812	0.606	0.714
Δ vs. 4B Base (LoRA)	+3.9	−2.4	+0.2	+2.4	−0.5	+0.9

592

Table 11: **Full ReSkip parameter-selection ablation.** Expanded version of Tab. 5. Scores are LAMBADA-500 accuracy for VLM rows and LIBERO 4-suite average success rate (%) for VLA rows. Δ is measured against the corresponding no-skip/full reference. For VLA, the reference is the Tab. 4 Full policy (97.2 for 2B and 96.7 for 4B). Rows labelled main-table use the operating points reported in the main experiments; VLA-style text probes are retained only as diagnostics.

Knob	Domain	Variant	$\mathcal{P}$	q	M	Score	Δ
<i>Eligible set $\mathcal{P}$</i>							
$\mathcal{P}$	VLM-2B	main-table (ours)	$\{4\}$	0.95	1	0.562	−0.2pp
$\mathcal{P}$	VLM-2B	VLA-style text probe	$\{1, 4\}$	0.85	2	0.560	−1.0pp
$\mathcal{P}$	VLM-2B	all eligible	all	0.99	4	0.454	−11.1pp
$\mathcal{P}$	VLM-4B	main-table (ours)	$\{1, 2\}$	0.85	2	0.616	−4.7pp
$\mathcal{P}$	VLM-4B	all eligible	all	0.99	4	0.542	−12.0pp
$\mathcal{P}$	VLA-2B	drift-selected	$\{1, 4\}$	0.99	2	97.35	+0.15pp
$\mathcal{P}$	VLA-4B	drift-selected	$\{1, 2\}$	0.99	2	96.45	−0.25pp
<i>Threshold quantile q</i>							
q	VLM-2B	main-table chosen	$\{4\}$	0.95	1	0.562	−0.2pp
q	VLM-2B	VLA-style no skip	$\{1, 4\}$	−	0	0.570	−
q	VLM-2B	VLA-style conservative	$\{1, 4\}$	0.85	2	0.560	−1.0pp
q	VLM-2B	VLA-style aggressive	$\{1, 4\}$	0.50	2	0.412	−15.8pp
q	VLM-2B	VLA-style aggressive	$\{1, 4\}$	0.30	2	0.390	−18.0pp
q	VLM-4B	main-table chosen	$\{1, 2\}$	0.85	2	0.616	−4.7pp
q	VLA-2B	full reference	$\{1, 4\}$	−	0	97.2	−
q	VLA-4B	full reference	$\{1, 2\}$	−	0	96.7	−
q	VLA-2B	over-skip	$\{1, 4\}$	0.85	2	83.1	−14.1pp
q	VLA-2B	knee	$\{1, 4\}$	0.95	2	94.7	−2.5pp
q	VLA-2B	chosen	$\{1, 4\}$	0.99	2	97.35	+0.15pp
q	VLA-4B	knee	$\{1, 2\}$	0.95	2	96.15	−0.55pp
q	VLA-4B	chosen	$\{1, 2\}$	0.99	2	96.45	−0.25pp
<i>Maximum skips $M_{\max}$</i>							
$M_{\max}$	VLM-2B	main cap	$\{4\}$	0.95	1	0.562	−0.2pp
$M_{\max}$	VLM-2B	VLA-style cap	$\{1, 4\}$	0.85	2	0.560	−1.0pp
$M_{\max}$	VLM-2B	wide cap	all	0.99	4	0.454	−11.1pp
$M_{\max}$	VLM-4B	main cap	$\{1, 2\}$	0.85	2	0.616	−4.7pp
$M_{\max}$	VLM-4B	cap saturation check	$\{1, 2\}$	0.85	$4 > \mathcal{P} $	0.616	−4.7pp
$M_{\max}$	VLM-4B	wide cap	all	0.99	4	0.542	−12.0pp
$M_{\max}$	VLA-2B	chosen cap	$\{1, 4\}$	0.99	2	97.35	+0.15pp
$M_{\max}$	VLA-2B	cap saturation check	$\{1, 4\}$	0.99	4	97.2	+0.0pp
$M_{\max}$	VLA-2B	wide cap	all	0.99	4	96.3	−0.9pp
$M_{\max}$	VLA-4B	chosen cap	$\{1, 2\}$	0.99	2	96.45	−0.25pp
$M_{\max}$	VLA-4B	cap saturation check	$\{1, 2\}$	0.99	4	96.3	−0.4pp
$M_{\max}$	VLA-4B	wide cap	all	0.99	4	89.5	−7.2pp

E VLA ReSkip calibration

The ReSkip eligible set $\mathcal{P}$ for each scale is selected from a per-block ablation on the trained retrofit-initialized OFT policy, not on the VLM retrofit alone, because the VLA action stream activates a different routing distribution than the language stream. Implementation diagnostics in this appendix use zero-based block ids, whereas the method equations use one-based block indices. We measure per-block action-drift MSE on a 24-sample simulated trajectory: the policy is run with no skip and with each block individually skipped, and the per-step action MSE between the two rollouts is the cost of removing that block on the action distribution. The two safest blocks per scale form $\mathcal{P}$.

2B policy per-block action-drift MSE (sorted ascending, in units $\times 10^{-3}$): block 1: 0.4; block 4: 34.0; block 0: 58.1; block 5: 87.0; remaining blocks > 100 . Selecting the two lowest: $\mathcal{P}_{2B} = \{1, 4\}$.

4B policy per-block action-drift MSE: block 1: 0.6; block 2: 11.0; block 0: 34.0; block 3: 63.0; remaining > 120 . Selecting: $\mathcal{P}_{4B} = \{1, 2\}$.

606 **Cross-scale transfer of $\mathcal{P}$ fails.** Naively transferring 2B’s $\mathcal{P}=\{1, 4\}$ onto 4B catastrophically drops
607 LIBERO-Spatial to 12.4% at $q=0.85$ — block-4’s drift on 4B is roughly $10\times$ that of block-2, so the
608 same eligible-set rule produces wildly different operating points across scales. $\mathcal{P}$ must be calibrated
609 on each model’s own sim distribution; this is the per-scale half of the modality-aware skip protocol.

610 **Threshold calibration.** Per-block thresholds τ_n are the q -th empirical quantile of $w_{\text{recent},n}$ over
611 31,286 (2B) / 31,257 (4B) sim-rollout records, one record per decoded action token. The VLA
612 q -sweep rows in Tab. 11 use this action-distribution calibration.

613 **Cross-modality threshold transfer is not safe.** The LAMBADA-calibrated thresholds at $q=0.85$
614 from Tab. 8, applied unchanged to the same trained policy at LIBERO inference, drop LIBERO-
615 Spatial from 99.5% to 64% in a partial evaluation. The action distribution has more low- w_{recent} mass
616 than the language distribution at the same blocks; text-calibrated thresholds therefore over-skip action
617 tokens. This motivates calibrating ReSkip on the deployment modality.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [\[Yes\]](#)

Justification: Claims are matched to the method and experiments in Sections 2–3; scope caveats are stated in the limitations checklist item below.

Guidelines:

- The answer [\[N/A\]](#) means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [\[No\]](#) or [\[N/A\]](#) answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [\[Yes\]](#)

Justification: The current study is limited to the Qwen3-VL model family, and the retrofit recipe depends on the SFT mix selected by the screening diagnostics; extending AR-Retrofit and ReSkip to other mainstream large-model families is left for future work. Wall-clock results are H100/bfloat16 only, VLA Pareto thresholds are calibrated per scale and deployment modality rather than universally transferable, and the method currently operates at block granularity rather than per-token granularity.

Guidelines:

- The answer [\[N/A\]](#) means that the paper has no limitation while the answer [\[No\]](#) means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [Yes]

Justification: The margin-preservation condition and assumptions are stated in Section 2.3.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: Reproduction details are provided in Sections 2–3 and Appendices A–B.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [No]

Justification: No anonymized package is included in the initial anonymous submission; code release is planned after de-anonymization, and the paper gives method, training, calibration, and evaluation details.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: Training, evaluation, and hyperparameter details are in Section 3 and Appendix A.1.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [No]

Justification: No formal multi-seed error bars, confidence intervals, or significance tests are reported. Robustness evidence comes from standard protocols, full lmms-eval splits, 500 LIBERO rollouts per suite, and appendix ablations.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.

- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: Appendix A.1 reports main retrofit training compute (4 H100 GPUs in bfloat16; about 15/33 minutes for Qwen3-VL-2B/4B, about 1.0/2.2 H100-hours); Section 3 and Appendix C report evaluation and latency settings.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The work uses existing models/benchmarks and does not involve human subjects or sensitive data collection.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: The work may reduce inference cost, but cheaper deployment can also lower misuse barriers for capable pretrained models.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: No new high-risk pretrained model, image generator, or scraped dataset is released.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: Existing assets are cited and their licenses/terms are summarized in Appendix A.2; no assets are redistributed or repackaged.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.

- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset’s creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [N/A]

Justification: No new dataset, benchmark, or public model asset is released in this submission.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: No crowdsourcing or human-subject experiments are involved.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [N/A]

Justification: No human-subject experiments are involved.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.

- 934 • Depending on the country in which research is conducted, IRB approval (or equivalent)
935 may be required for any human subjects research. If you obtained IRB approval, you
936 should clearly state this in the paper.
- 937 • We recognize that the procedures for this may vary significantly between institutions
938 and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the
939 guidelines for their institution.
- 940 • For initial submissions, do not include any information that would break anonymity (if
941 applicable), such as the institution conducting the review.

942 16. Declaration of LLM usage

943 Question: Does the paper describe the usage of LLMs if it is an important, original, or
944 non-standard component of the core methods in this research? Note that if the LLM is used
945 only for writing, editing, or formatting purposes and does *not* impact the core methodology,
946 scientific rigor, or originality of the research, declaration is not required.

947 Answer: [N/A]

948 Justification: The work studies pretrained LLM/VLM/VLA backbones, but no LLM judge,
949 synthetic-labeling pipeline, or other non-standard LLM component is used in the core
950 method or evaluation pipeline.

951 Guidelines:

- 952 • The answer [N/A] means that the core method development in this research does not
953 involve LLMs as any important, original, or non-standard components.
- 954 • Please refer to our LLM policy in the NeurIPS handbook for what should or should not
955 be described.